

Real Time Analytics

YUNATA GUNAWAN

DEC. 2024 – JAN. 2025



Use Case

MOVIE STREAM TRANSACTION

Tuning Pulsar JVM

\$PULSAR_HOME/CONF/BKENV.SH

```
GNU nano 6.2 bkenv.sh
# Memory size options
BOOKIE_MEM=${BOOKIE_MEM:-${PULSAR_MEM:-"-Xms2g -Xmx2g -XX:MaxDirectMemorySize=2g"}}

if [ -z "$JAVA_HOME" ]; then
    JAVA_BIN=java
else
    JAVA_BIN="$JAVA_HOME/bin/java"
fi
for token in $($JAVA_BIN -version 2>&1 | grep 'version '); do
    if [[ $token =~ \("[[:digit:]]+\)\.([[:digit:]]+)(.*)" ]]; then
        if [[ ${BASH_REMATCH[1]} == "1" ]]; then
            JAVA_MAJOR_VERSION=${BASH_REMATCH[2]}
        else
            JAVA_MAJOR_VERSION=${BASH_REMATCH[1]}
        fi
        break
    elif [[ $token =~ \("[[:digit:]]+)(.*)" ]]; then
        # Process the java versions without dots, such as `17-internal`.
        JAVA_MAJOR_VERSION=${BASH_REMATCH[1]}
        break
    fi
done

# Garbage collection options
BOOKIE_GC=${BOOKIE_GC:-${PULSAR_GC}}
if [ -z "$BOOKIE_GC" ]; then
    BOOKIE_GC="-XX:+PerfDisableSharedMem -XX:+AlwaysPreTouch"
    if [[ $JAVA_MAJOR_VERSION -ge 21 ]]; then
        BOOKIE_GC="-XX:+UseZGC -XX:+ZGenerational ${BOOKIE_GC}"
    fi
fi
```

\$PULSAR_HOME/CONF/PULSAR_ENV.SH

```
GNU nano 6.2 pulsar_env.sh
# Extra options to be passed to the jvm
PULSAR_MEM=${PULSAR_MEM:-"-Xms2g -Xmx2g -XX:MaxDirectMemorySize=4g"}

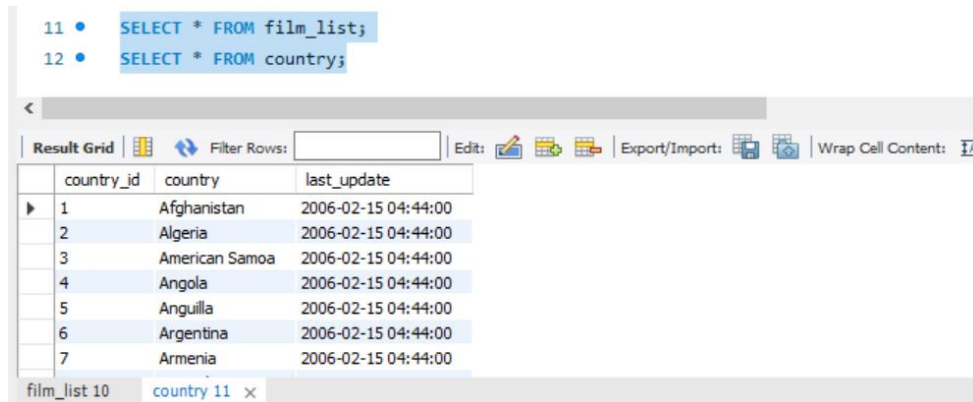
if [ -z "$JAVA_HOME" ]; then
    JAVA_BIN=java
else
    JAVA_BIN="$JAVA_HOME/bin/java"
fi
for token in $($JAVA_BIN -version 2>&1 | grep 'version '); do
    if [[ $token =~ \("[[:digit:]]+\)\.([[:digit:]]+)(.*)" ]]; then
        if [[ ${BASH_REMATCH[1]} == "1" ]]; then
            JAVA_MAJOR_VERSION=${BASH_REMATCH[2]}
        else
            JAVA_MAJOR_VERSION=${BASH_REMATCH[1]}
        fi
        break
    elif [[ $token =~ \("[[:digit:]]+)(.*)" ]]; then
        # Process the java versions without dots, such as `17-internal`.
        JAVA_MAJOR_VERSION=${BASH_REMATCH[1]}
        break
    fi
done

# Garbage collection options
if [ -z "$PULSAR_GC" ]; then
    PULSAR_GC="-XX:+PerfDisableSharedMem -XX:+AlwaysPreTouch"
    if [[ $JAVA_MAJOR_VERSION -ge 21 ]]; then
        PULSAR_GC="-XX:+UseZGC -XX:+ZGenerational ${PULSAR_GC}"
    else
        PULSAR_GC="-XX:+UseZGC ${PULSAR_GC}"
    fi
fi
```

Table Used

USE sakila;

```
CREATE TABLE movie_stream (  
  ID BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  BATCH_DATE DATE NOT NULL,  
  SESSION_ID CHAR(36) NOT NULL,  
  COUNTRY_FID INT NOT NULL,  
  FILM_FID INT NOT NULL,  
  PLATFORM CHAR(7) NOT NULL  
);
```



The screenshot shows a database client interface. At the top, two SQL queries are listed: `SELECT * FROM film_list;` and `SELECT * FROM country;`. Below the queries is a 'Result Grid' tab. The grid displays the results of the second query, showing a list of countries with their IDs and last update times. The columns are 'country_id', 'country', and 'last_update'. The data rows are numbered 1 through 7, corresponding to the countries listed. The interface also includes a 'Filter Rows' field, an 'Edit' button, and an 'Export/Import' button.

country_id	country	last_update
1	Afghanistan	2006-02-15 04:44:00
2	Algeria	2006-02-15 04:44:00
3	American Samoa	2006-02-15 04:44:00
4	Angola	2006-02-15 04:44:00
5	Anguilla	2006-02-15 04:44:00
6	Argentina	2006-02-15 04:44:00
7	Armenia	2006-02-15 04:44:00

```
CREATE TABLE sakila.film_poc_table (  
  ID BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  FILM_NAME CHAR(50) NOT NULL,  
  YEAR INT NOT NULL,  
  GENRE CHAR(20) NOT NULL  
);
```

```
insert into sakila.film_poc_table values (1, 'Mufasa: The Lion  
King', 2024, 'Adventure');
```

\$PULSAR_HOME/connectors/mysql.yaml

```
tenant: "public"
namespace: "default"
name: "debezium-mysql-source"
topicName: "debezium-mysql-topic"
archive: "connectors/pulsar-io-debezium-mysql-4.0.1.nar"

parallelism: 1

configs:
  ## config for mysql, docker image: debezium/example-mysql:0.8
  database.hostname: "192.168.240.29"
  database.port: "3306"
  database.user: "admin"
  database.password: "admin"
  database.server.id: "184054"
  database.server.name: "mysql"
  database.whitelist: "sakila"
  database.history: "org.apache.pulsar.io.debezium.PulsarDatabaseHistory"
  database.history.pulsar.topic: "history-topic"
  database.history.pulsar.service.url: "pulsar://127.0.0.1:6650"

  ## KEY_CONVERTER_CLASS_CONFIG, VALUE_CONVERTER_CLASS_CONFIG
  key.converter: "org.apache.kafka.connect.json.JsonConverter"
  value.converter: "org.apache.kafka.connect.json.JsonConverter"

  ## PULSAR_SERVICE_URL_CONFIG
  pulsar.service.url: "pulsar://127.0.0.1:6650"

  ## OFFSET_STORAGE_TOPIC_CONFIG
  offset.storage.topic: "offset-topic"
```

```
tenant: "public"
namespace: "default"
name: "debezium-mysql-source"
topicName: "debezium-mysql-topic"
archive: "connectors/pulsar-io-debezium-mysql-4.0.1.nar"

parallelism: 1

configs:
  ## config for mysql, docker image: debezium/example-mysql:0.8
  database.hostname: "192.168.240.29"
  database.port: "3306"
  database.user: "admin"
  database.password: "admin"
  database.server.id: "184054"
  database.server.name: "mysql"
  database.whitelist: "sakila"
  database.history: "org.apache.pulsar.io.debezium.PulsarDatabaseHistory"
  database.history.pulsar.topic: "history-topic"
  database.history.pulsar.service.url: "pulsar://127.0.0.1:6650"

  ## KEY_CONVERTER_CLASS_CONFIG, VALUE_CONVERTER_CLASS_CONFIG
  key.converter: "org.apache.kafka.connect.json.JsonConverter"
  value.converter: "org.apache.kafka.connect.json.JsonConverter"

  ## PULSAR_SERVICE_URL_CONFIG
  pulsar.service.url: "pulsar://127.0.0.1:6650"

  ## OFFSET_STORAGE_TOPIC_CONFIG
  offset.storage.topic: "offset-topic"
```

MySQL Stored Procedure Script (3 Hour Ingestion Every 5 Second)

DELIMITER \$\$

CREATE PROCEDURE InsertRandomDataFromDate(IN batch_date DATE)

BEGIN

DECLARE end_time DATETIME;
DECLARE random_country_fid INT;
DECLARE random_film_fid INT;
DECLARE random_platform CHAR(7);
DECLARE random_session_id CHAR(36);

-- Calculate the end time (3 hours after the current time)
SET end_time = DATE_ADD(NOW(), INTERVAL 3 HOUR);

-- Start the loop

WHILE NOW() < end_time DO

-- Generate a random country_fid (for example, between 1 and 109)
SET random_country_fid = FLOOR(1 + (RAND() * 109));

-- Generate a random film_fid (for example, between 1 and 1000)
SET random_film_fid = FLOOR(1 + (RAND() * 1000));

-- Generate a random platform (e.g., 'Netflix', 'Hulu', 'Disney')
SET random_platform = ELT(FLOOR(1 + (RAND() * 3)), 'Netflix', 'Hulu', 'Disney');

-- Generate a random session_id (UUID)
SET random_session_id = UUID();

-- Insert the random data into the movie_stream_data table (session_id first)
INSERT INTO movie_stream_data (SESSION_ID, BATCH_DATE, COUNTRY_FID, FILM_FID, PLATFORM)
VALUES (random_session_id, batch_date, random_country_fid, random_film_fid, random_platform);

-- Print the message with the time and inserted values
SELECT CONCAT('Inserted session_id: ', random_session_id,
 '| country_fid: ', random_country_fid,
 '| film_fid: ', random_film_fid,
 '| platform: ', random_platform,
 '| Time: ', NOW()) AS ConfirmationMessage;

-- Wait for 5 seconds before the next insert
DO SLEEP(5);

END WHILE;
END\$\$

DELIMITER ;

CALL InsertRandomDataFromDate('2024-12-22');

MySQL Server Debezium CDC Result

```
$PULSAR_HOME/bin/pulsar-admin source localrun --sourceConfigFile $PULSAR_HOME/debezium/mysql.yaml
```

```
$PULSAR_HOME/bin/pulsar-client consume persistent://public/default/mysql.sakila.poc_test --num-messages 0 --subscription-name mysql-sub --subscription-type Shared -p Earliest
```

[Insert Query Result of CDC]

```
----- got message -----
publishTime:[1734871768842], eventTime:[0], key:[null], properties:[], content:{"before":null,"after":{"id":1,"name":"john doe"},"source":{"version":"1.9.7.Final","connector":"mysql","name":"mysql","ts_ms":1734871766000,"snapshot":"true","db":"sakila","sequence":null,"table":"poc_test","server_id":0,"gtid":null,"file":"PCDEV-bin.000004","pos":1293,"row":0,"thread":null,"query":null},"op":"r","ts_ms":1734871767032,"transaction":null}
----- got message -----
publishTime:[1734871768842], eventTime:[0], key:[null], properties:[], content:{"before":null,"after":{"id":2,"name":"Maria"},"source":{"version":"1.9.7.Final","connector":"mysql","name":"mysql","ts_ms":1734871766000,"snapshot":"true","db":"sakila","sequence":null,"table":"poc_test","server_id":0,"gtid":null,"file":"PCDEV-bin.000004","pos":1293,"row":0,"thread":null,"query":null},"op":"r","ts_ms":1734871767032,"transaction":null}
----- got message -----
publishTime:[1734871768842], eventTime:[0], key:[null], properties:[], content:{"before":null,"after":{"id":3,"name":"Maria"},"source":{"version":"1.9.7.Final","connector":"mysql","name":"mysql","ts_ms":1734871766000,"snapshot":"true","db":"sakila","sequence":null,"table":"poc_test","server_id":0,"gtid":null,"file":"PCDEV-bin.000004","pos":1293,"row":0,"thread":null,"query":null},"op":"r","ts_ms":1734871767032,"transaction":null}
----- got message -----
publishTime:[1734871862878], eventTime:[0], key:[null], properties:[], content:{"before":null,"after":{"id":4,"name":"Maria"},"source":{"version":"1.9.7.Final","connector":"mysql","name":"mysql","ts_ms":1734871861000,"snapshot":"false","db":"sakila","sequence":null,"table":"poc_test","server_id":1,"gtid":null,"file":"PCDEV-bin.000004","pos":1513,"row":0,"thread":10,"query":null},"op":"c","ts_ms":1734871861952,"transaction":null}
```

[Update & Delete Query Result CDC]

```
----- got message -----
publishTime:[1734872331560], eventTime:[0], key:[null], properties:[], content:{"before":{"id":1,"name":"john doe"},"after":{"id":1,"name":"Lara"},"source":{"version":"1.9.7.Final","connector":"mysql","name":"mysql","ts_ms":1734872330000,"snapshot":"false","db":"sakila","sequence":null,"table":"poc_test","server_id":1,"gtid":null,"file":"PCDEV-bin.000004","pos":2120,"row":0,"thread":42,"query":null},"op":"u","ts_ms":1734872330959,"transaction":null}
----- got message -----
publishTime:[1734872356159], eventTime:[0], key:[null], properties:[], content:{"before":{"id":3,"name":"Maria"},"after":{"id":3,"name":"Kira"},"source":{"version":"1.9.7.Final","connector":"mysql","name":"mysql","ts_ms":1734872354000,"snapshot":"false","db":"sakila","sequence":null,"table":"poc_test","server_id":1,"gtid":null,"file":"PCDEV-bin.000004","pos":2441,"row":0,"thread":42,"query":null},"op":"u","ts_ms":1734872355489,"transaction":null}
----- got message -----
publishTime:[1734872368718], eventTime:[0], key:[null], properties:[], content:{"before":{"id":3,"name":"Kira"},"after":null,"source":{"version":"1.9.7.Final","connector":"mysql","name":"mysql","ts_ms":1734872367000,"snapshot":"false","db":"sakila","sequence":null,"table":"poc_test","server_id":1,"gtid":null,"file":"PCDEV-bin.000004","pos":2750,"row":0,"thread":42,"query":null},"op":"d","ts_ms":1734872368175,"transaction":null}
2024-12-22T12:59:33.431+0000 [pulsar-timer-16-1] INFO org.apache.pulsar.client.impl.ConsumerStatsRecorderImpl - [persistent://public/default/mysql.sakila.poc_test] [mysql-sub] [hkoVP] Prefetched messages: 0 --- Consume throughput received: 0.05 msgs/s --- 0.00 Mbit/s --- Ack sent rate: 0.05 ack/s --- Failed messages: 0 --- batch messages: 0 ---Failed acks: 0
```

<https://debezium.io/blog/2019/05/23/tutorial-using-debezium-connectors-with-apache-pulsar/>
<https://pulsar.apache.org/docs/4.0.x/io-cdc-debezium/>

The Result Topics Is Not Visible At First, You Must Consume It

```
HDP.tlp - hdadmin@192.168.29.221:22 - Bitvise xterm - hdadmin@analytic: ~
0x91f85e0c, L:/127.0.0.1:56948 - R:localhost/127.0.0.1:6650]
2024-12-27T05:04:03,736+0000 [pulsar-client-io-1-3] INFO org.apache.pulsar.io.kafka.connect.AbstractKafkaConnectSource - Finished flushing offsets to storage
2024-12-27T05:04:06,231+0000 [pulsar-timer-16-1] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - [public/default/debezium-mssql-source-test2-debezium-offset-topic] [standalone-658-38]
--- Publish throughput: 0.02 msg/s --- 0.00 Mbit/s --- Latency: med: 10.000 ms - 95pct: 10.000 ms - 99pct: 10.000 ms - 99.9pct: 10.000 ms - max: 10.000 ms --- BatchSize: med: 1.000 - 95pct: 1.000 - 99pct: 1.000 - 99.9pct: 1.000 - max: 1.000 --- MsgSize: med: 119.000 bytes - 95pct: 119.000 bytes - 99pct: 119.000 bytes - 99.9pct: 119.000 bytes - max: 119.000 bytes --- Ack received rate: 0.02 ack/s --- Failed messages: 0 --- Pending messages: 0
2024-12-27T05:04:06,312+0000 [pulsar-timer-16-1] INFO org.apache.pulsar.client.impl.ConsumerStatsRecorderImpl - [public/default/debezium-mssql-source-test2-debezium-offset-topic] [reader-e8c754da54] [WjdGS] Prefetched messages: 0 --- Consume throughput received: 0.10 msg/s --- 0.00 Mbit/s --- Ack sent rate: 0.00 ack/s --- Failed messages: 0 --- batch messages: 0 --- Failed acks: 0
2024-12-27T05:04:13,283+0000 [pulsar-client-io-1-3] INFO org.apache.pulsar.io.kafka.connect.AbstractKafkaConnectSource - Finished flushing offsets to storage
2024-12-27T05:05:03,679+0000 [pulsar-timer-16-1] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - [persistent://public/default/mssql.dbo.poc_test] [standalone-658-39] --- Publish throughput: 0.03 msg/s --- 0.00 Mbit/s --- Latency: med: 20.000 ms - 95pct: 20.000 ms - 99pct: 20.000 ms - 99.9pct: 20.000 ms - max: 20.000 ms --- BatchSize: med: 1.000 - 95pct: 1.000 - 99pct: 1.000 - 99.9pct: 1.000 - max: 1.000 --- MsgSize: med: 349.000 bytes - 95pct: 349.000 bytes - 99pct: 349.000 bytes - 99.9pct: 349.000 bytes - max: 349.000 bytes --- Ack received rate: 0.03 ack/s --- Failed messages: 0 --- Pending messages: 0
2024-12-27T05:05:06,234+0000 [pulsar-timer-16-1] INFO org.apache.pulsar.client.impl.ProducerStatsRecorderImpl - [public/default/debezium-mssql-source-test2-debezium-offset-topic] [standalone-658-38]
--- Publish throughput: 0.02 msg/s --- 0.00 Mbit/s --- Latency: med: 6.000 ms - 95pct: 6.000 ms - 99pct: 6.000 ms - 99.9pct: 6.000 ms - max: 6.000 ms --- BatchSize: med: 1.000 - 95pct: 1.000 - 99pct: 1.000 - 99.9pct: 1.000 - max: 1.000 --- MsgSize: med: 119.000 bytes - 95pct: 119.000 bytes - 99pct: 119.000 bytes - 99.9pct: 119.000 bytes - max: 119.000 bytes --- Ack received rate: 0.02 ack/s --- Failed messages: 0 --- Pending messages: 0
2024-12-27T05:05:06,315+0000 [pulsar-timer-16-1] INFO org.apache.pulsar.client.impl.ConsumerStatsRecorderImpl - [public/default/debezium-mssql-source-test2-debezium-offset-topic] [reader-e8c754da54] [WjdGS] Prefetched messages: 0 --- Consume throughput received: 0.02 msg/s --- 0.00 Mbit/s --- Ack sent rate: 0.00 ack/s --- Failed messages: 0 --- batch messages: 0 --- Failed acks: 0
```

```
HDP.tlp - hdadmin@192.168.29.221:22 - Bitvise xterm - hdadmin@analytic: ~
- [persistent://public/default/mssql.dbo.poc_test][sub] Subscribing to topic on cnx [id: 0x99c4f118, L:/127.0.0.1:57288 - R:localhost/127.0.0.1:6650], consumerId 0
2024-12-27T05:06:18,273+0000 [pulsar-client-io-1-3] INFO org.apache.pulsar.client.impl.ConsumerImpl - [persistent://public/default/mssql.dbo.poc_test][sub] Subscribed to topic on localhost/127.0.0.1:6650 -- consumer: 0
----- got message -----
publishTime:[1735275768714], eventTime:[0], key:[eyJpZCI6Mn0=], properties:[], content:{"before":null,"after":{"id":2,"name":"lorem","is_married":false},"source":{"version":"1.9.7.Final","connector":"sqlserver","name":"mssql","ts_ms":1735275765407,"snapshot":"false","db":"AW2022_SRC","sequence":null,"schema":"dbo","table":"poc_test","change_lsn":"00000037:00000538:0002","commit_lsn":"00000037:00000538:0003","event_serial_no":1},"op":"c","ts_ms":1735275767955,"transaction":null}}
----- got message -----
publishTime:[1735275843702], eventTime:[0], key:[eyJpZCI6M30=], properties:[], content:{"before":null,"after":{"id":3,"name":"lorem","is_married":false},"source":{"version":"1.9.7.Final","connector":"sqlserver","name":"mssql","ts_ms":1735275836753,"snapshot":"false","db":"AW2022_SRC","sequence":null,"schema":"dbo","table":"poc_test","change_lsn":"00000037:000005c0:0002","commit_lsn":"00000037:000005c0:0003","event_serial_no":1},"op":"c","ts_ms":1735275843022,"transaction":null}}
----- got message -----
publishTime:[1735275853261], eventTime:[0], key:[eyJpZCI6NH0=], properties:[], content:{"before":null,"after":{"id":4,"name":"lorem","is_married":false},"source":{"version":"1.9.7.Final","connector":"sqlserver","name":"mssql","ts_ms":1735275848880,"snapshot":"false","db":"AW2022_SRC","sequence":null,"schema":"dbo","table":"poc_test","change_lsn":"00000037:000005e0:0002","commit_lsn":"00000037:000005e0:0003","event_serial_no":1},"op":"c","ts_ms":1735275853009,"transaction":null}}
----- got message -----
publishTime:[1735275984468], eventTime:[0], key:[eyJpZCI6NX0=], properties:[], content:{"before":null,"after":{"id":5,"name":"lorem","is_married":false},"source":{"version":"1.9.7.Final","connector":"sqlserver","name":"mssql","ts_ms":1735275980850,"snapshot":"false","db":"AW2022_SRC","sequence":null,"schema":"dbo","table":"poc_test","change_lsn":"00000037:000006c0:0002","commit_lsn":"00000037:000006c0:0003","event_serial_no":1},"op":"c","ts_ms":1735275983506,"transaction":null}}
```

Reading From Debezium

<https://pulsar.apache.org/docs/4.0.x/io-cdc-debezium/>

```
$PULSAR_HOME/bin/pulsar-admin source localrun --sourceConfigFile
```

```
$PULSAR_HOME/debezium/mysql.yaml
```

```
$PULSAR_HOME/bin/pulsar-client consume
```

```
persistent://public/default/mysql.sakila.movie_stream --num-messages 0 --subscription-  
name debezium-subscription --subscription-type Shared -p Earliest
```

```
$PULSAR_HOME/bin/pulsar-client consume persistent://public/default/avro-movie_stream --num-  
messages 0 --subscription-name avro-subscription --subscription-type Shared -p Earliest
```

Generate Maven Project

```
mvn archetype:generate -DarchetypeGroupId=org.apache.maven.archetypes -  
DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.5
```

Template Reading From Debezium

```
package pipeline;

import org.apache.pulsar.client.api.*;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

public class PulsarConsumerDebeziumByte {
    public static void main(String[] args) throws PulsarClientException {
        PulsarClient client = PulsarClient.builder()
            .serviceUrl("pulsar://localhost:6650")
            .build();

        Consumer<byte[]> consumer = client.newConsumer()
            .topic("persistent://public/default/mysql.sakila.movie_stream")
            .subscriptionName("debezium-subscription")
            .subscriptionType(SubscriptionType.Shared)
            .subscribe();

        ObjectMapper objectMapper = new ObjectMapper();

        while (true) {
            Message<byte[]> msg = consumer.receive();
            try {
                // System.out.printf("Message received: %s\n", new String(msg.getData()));
                JsonNode rootNode = objectMapper.readTree(new String(msg.getData()));
                char op = rootNode.get("op").toString().charAt(1);

                // Filter only insert and update CDC
                if (op != 'd') {
                    JsonNode afterNode = rootNode.get("after");

                    // (Optional) Push the message to specific partition
                }

                // Acknowledge the message but not processing the delete operation
                consumer.acknowledge(msg);
            } catch (Exception e) {
                //e.printStackTrace();
                consumer.negativeAcknowledge(msg);
            }
        }
    }
}
```

GlobalParam.java

```
package parameter;

public class GlobalParam {
    public static String ipAddress = "192.168.29.221";
    public static String brokerIp = "pulsar://" + ipAddress + ":6650";
    public static String pulsarWebService = "http://" + ipAddress + ":8080";
    public static String defaultTenant = "public";
    public static String defaultNamespace = "default";
    public static String defaultSubscription = "debezium-subscription";

    public static String flinkBroker = "";

    public static String generateTopicFQN (int isPersistent, String tenant, String namespace, String topic) {
        String persistent = (isPersistent == 1) ? "persistent": "non-persistent";
        return persistent + "://" + tenant + "/" + namespace + "/" + topic;
    }
}
```

MovieStream.java

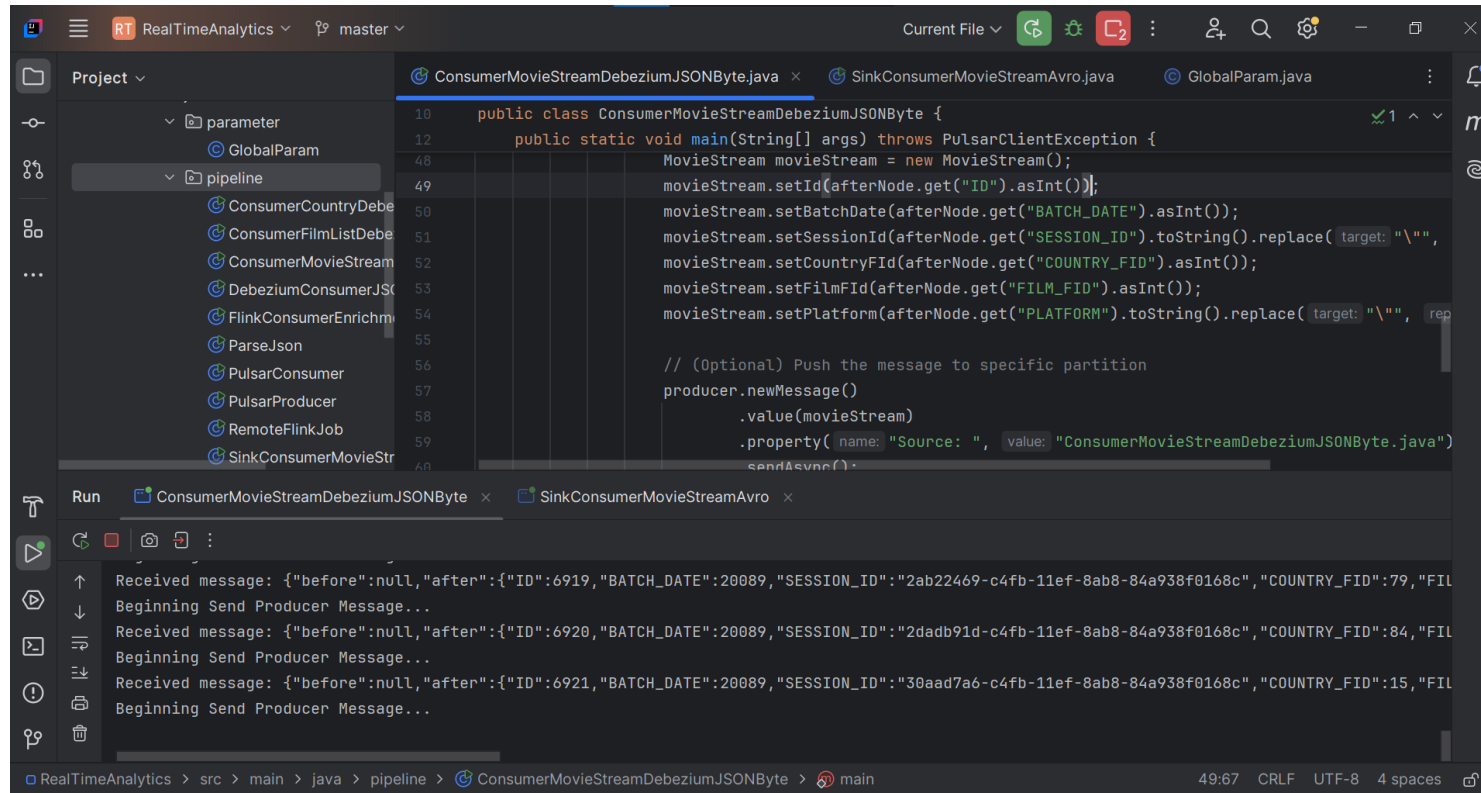
```
package schema;

public class MovieStream {
    private int id;
    private int batchDate;
    private String sessionId;
    private int countryFId;
    private int filmFId;
    private String platform;

    // No-args Constructor is Mandatory
    public MovieStream() {
    }

    // Setter and Getter Methods: For presentation is not showed ..
}
```

Debezium Result In Java Client Consumer and Send to Another Topic



The screenshot displays an IDE window for a project named "RealTimeAnalytics". The left sidebar shows a project tree with a "pipeline" folder containing several Java classes, including "ConsumerMovieStreamDebeziumJSONByte.java". The main editor area shows the code for this class, which is a public class with a static void main method. The code uses the Debezium API to consume messages from a topic and send them to another topic. The code is as follows:

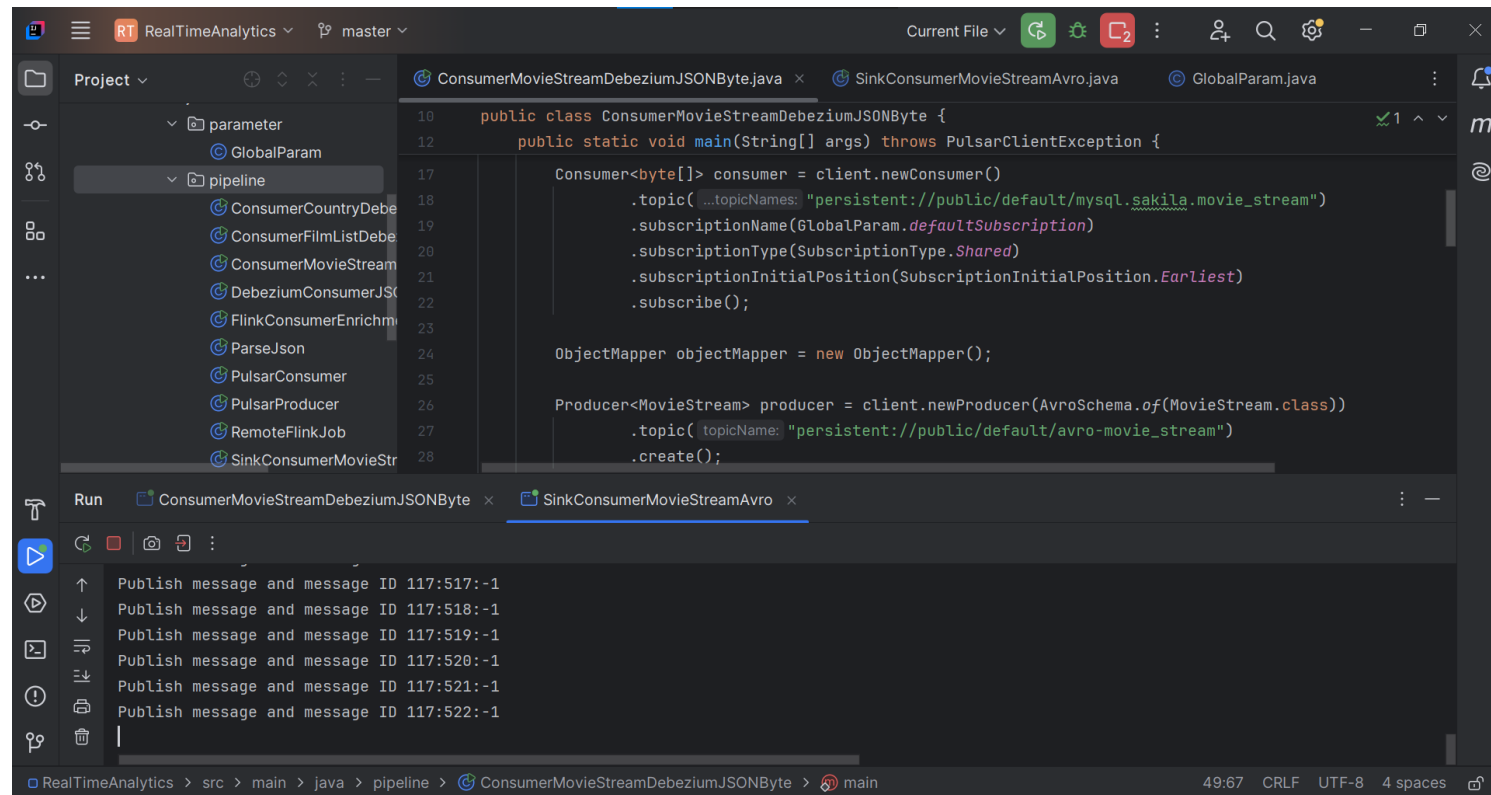
```
10 public class ConsumerMovieStreamDebeziumJSONByte {
11     public static void main(String[] args) throws PulsarClientException {
48         MovieStream movieStream = new MovieStream();
49         movieStream.setId(afterNode.get("ID").asInt());
50         movieStream.setBatchDate(afterNode.get("BATCH_DATE").asInt());
51         movieStream.setSessionId(afterNode.get("SESSION_ID").toString().replace( target: "\"",
52         movieStream.setCountryFid(afterNode.get("COUNTRY_FID").asInt());
53         movieStream.setFilmFid(afterNode.get("FILM_FID").asInt());
54         movieStream.setPlatform(afterNode.get("PLATFORM").toString().replace( target: "\"", rep
55
56         // (Optional) Push the message to specific partition
57         producer.newMessage()
58             .value(movieStream)
59             .property( name: "Source: ", value: "ConsumerMovieStreamDebeziumJSONByte.java")
60             .sendAsync();
```

The bottom panel shows the "Run" output, which displays the received messages and the beginning of the send producer message process. The messages are as follows:

```
Received message: {"before":null,"after":{"ID":6919,"BATCH_DATE":20089,"SESSION_ID":"2ab22469-c4fb-11ef-8ab8-84a938f0168c","COUNTRY_FID":79,"FIL
Beginning Send Producer Message...
Received message: {"before":null,"after":{"ID":6920,"BATCH_DATE":20089,"SESSION_ID":"2dad91d-c4fb-11ef-8ab8-84a938f0168c","COUNTRY_FID":84,"FIL
Beginning Send Producer Message...
Received message: {"before":null,"after":{"ID":6921,"BATCH_DATE":20089,"SESSION_ID":"30aad7a6-c4fb-11ef-8ab8-84a938f0168c","COUNTRY_FID":15,"FIL
Beginning Send Producer Message...
```

The bottom status bar shows the file path: "RealTimeAnalytics > src > main > java > pipeline > ConsumerMovieStreamDebeziumJSONByte > main". The status bar also shows the file encoding: "UTF-8" and the line length: "49:67".

Receive Data and Push To Sink (Both Using Avro Serdes)



The screenshot shows an IDE window with the following components:

- Project Explorer:** Shows a project structure with a `pipeline` folder containing several Java files, including `ConsumerMovieStreamDebeziumJSONByte.java`.
- Editor:** Displays the code for `ConsumerMovieStreamDebeziumJSONByte.java`. The code is as follows:

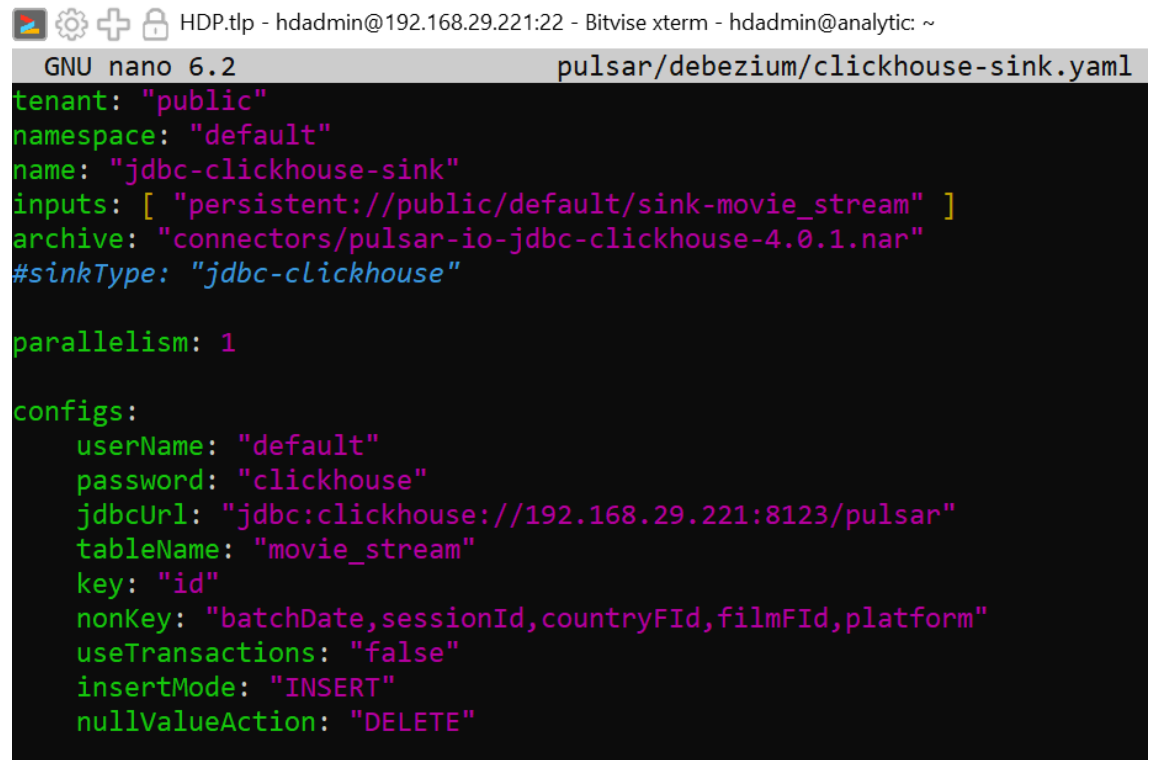
```
10 public class ConsumerMovieStreamDebeziumJSONByte {
11     public static void main(String[] args) throws PulsarClientException {
12
13         Consumer<byte[]> consumer = client.newConsumer()
14             .topic(...topicNames: "persistent://public/default/mysql.sakila.movie_stream")
15             .subscriptionName(GlobalParam.defaultSubscription)
16             .subscriptionType(SubscriptionType.Shared)
17             .subscriptionInitialPosition(SubscriptionInitialPosition.Earliest)
18             .subscribe();
19
20         ObjectMapper objectMapper = new ObjectMapper();
21
22         Producer<MovieStream> producer = client.newProducer(AvroSchema.of(MovieStream.class))
23             .topic(topicName: "persistent://public/default/avro-movie_stream")
24             .create();
```
- Run Console:** Shows the output of the program, which consists of six lines of log messages: "Publish message and message ID 117:517:-1", "Publish message and message ID 117:518:-1", "Publish message and message ID 117:519:-1", "Publish message and message ID 117:520:-1", "Publish message and message ID 117:521:-1", and "Publish message and message ID 117:522:-1".
- Bottom Bar:** Shows the file path `RealTimeAnalytics > src > main > java > pipeline > ConsumerMovieStreamDebeziumJSONByte > main`, the current line number `49:67`, and the encoding `CRLF UTF-8 4 spaces`.

Create Clickhouse Table And Configuration

```
CREATE DATABASE pulsar;
```

```
USE pulsar;
```

```
CREATE TABLE movie_stream (id INT NOT NULL, batchDate INT NOT NULL, sessionId String NOT NULL, countryFId INT NOT NULL, filmFId INT NOT NULL, platform String NOT NULL) Engine=MergeTree() PRIMARY KEY (id);
```



```
GNU nano 6.2 pulsar/debezium/clickhouse-sink.yaml
tenant: "public"
namespace: "default"
name: "jdbc-clickhouse-sink"
inputs: [ "persistent://public/default/sink-movie_stream" ]
archive: "connectors/pulsar-io-jdbc-clickhouse-4.0.1.nar"
#sinkType: "jdbc-clickhouse"

parallelism: 1

configs:
  username: "default"
  password: "clickhouse"
  jdbcUrl: "jdbc:clickhouse://192.168.29.221:8123/pulsar"
  tableName: "movie_stream"
  key: "id"
  nonKey: "batchDate,sessionId,countryFId,filmFId,platform"
  useTransactions: "false"
  insertMode: "INSERT"
  nullValueAction: "DELETE"
```

Running The Sink Connector and Result In Clickhouse

```
$PULSAR_HOME/bin/pulsar-admin sink localrun --sinkConfigFile  
$PULSAR_HOME/debezium/clickhouse-sink.yaml
```

HDP.tlp - hdadmin@192.168.29.221:22 - Bitvise xterm - hdadmin@analytic: ~/kafka

```
141 rows in set. Elapsed: 0.004 sec.  
analytic :) select * from movie_stream;  
SELECT *  
FROM movie_stream  
Query id: 58e1e7fa-f706-44fc-92b9-9322ca405f07
```

	id	batchDate	sessionId	countryFid	filmFid	platform
1.	6629	20089	c83943aa-c4f7-11ef-8ab8-84a938f0168c	23	33	Hulu
2.	6630	20089	cb36612d-c4f7-11ef-8ab8-84a938f0168c	72	641	Netflix
3.	6631	20089	ce338feb-c4f7-11ef-8ab8-84a938f0168c	24	383	Netflix
4.	6632	20089	d13151fc-c4f7-11ef-8ab8-84a938f0168c	25	283	Disney
5.	6633	20089	d42eb01b-c4f7-11ef-8ab8-84a938f0168c	98	240	Hulu
6.	6634	20089	d72bf33-c4f7-11ef-8ab8-84a938f0168c	89	525	Netflix
7.	6635	20089	da27c2d1-c4f7-11ef-8ab8-84a938f0168c	46	484	Netflix
8.	6636	20089	dd23a32c-c4f7-11ef-8ab8-84a938f0168c	44	487	Netflix
9.	6637	20089	e020da97-c4f7-11ef-8ab8-84a938f0168c	79	891	Netflix
10.	6638	20089	e31e0e28-c4f7-11ef-8ab8-84a938f0168c	87	86	Netflix
11.	6639	20089	e61b4330-c4f7-11ef-8ab8-84a938f0168c	109	838	Netflix
12.	6640	20089	e9189763-c4f7-11ef-8ab8-84a938f0168c	51	714	Netflix
13.	6641	20089	ec148b3a-c4f7-11ef-8ab8-84a938f0168c	88	461	Disney
14.	6642	20089	ef11ac44-c4f7-11ef-8ab8-84a938f0168c	6	589	Disney

HDP.tlp - hdadmin@192.168.29.221:22 - Bitvise xterm - hdadmin@analytic: ~/kafka

```
SELECT *  
FROM movie_stream  
Query id: 66810fb7-51d3-4ed8-8f32-530d0be06801
```

	id	batchDate	sessionId	countryFid	filmFid	platform
1.	6655	20089	15ec14b6-c4f8-11ef-8ab8-84a938f0168c	65	70	Hulu
2.	6656	20089	18e9271d-c4f8-11ef-8ab8-84a938f0168c	75	690	Hulu
3.	6657	20089	1be656d4-c4f8-11ef-8ab8-84a938f0168c	98	278	Disney
4.	6629	20089	c83943aa-c4f7-11ef-8ab8-84a938f0168c	23	33	Hulu
5.	6630	20089	cb36612d-c4f7-11ef-8ab8-84a938f0168c	72	641	Netflix
6.	6631	20089	ce338feb-c4f7-11ef-8ab8-84a938f0168c	24	383	Netflix
7.	6632	20089	d13151fc-c4f7-11ef-8ab8-84a938f0168c	25	283	Disney
8.	6633	20089	d42eb01b-c4f7-11ef-8ab8-84a938f0168c	98	240	Hulu
9.	6634	20089	d72bf33-c4f7-11ef-8ab8-84a938f0168c	89	525	Netflix
10.	6635	20089	da27c2d1-c4f7-11ef-8ab8-84a938f0168c	46	484	Netflix
11.	6636	20089	dd23a32c-c4f7-11ef-8ab8-84a938f0168c	44	487	Netflix
12.	6637	20089	e020da97-c4f7-11ef-8ab8-84a938f0168c	79	891	Netflix
13.	6638	20089	e31e0e28-c4f7-11ef-8ab8-84a938f0168c	87	86	Netflix
14.	6639	20089	e61b4330-c4f7-11ef-8ab8-84a938f0168c	109	838	Netflix
15.	6640	20089	e9189763-c4f7-11ef-8ab8-84a938f0168c	51	714	Netflix
16.	6641	20089	ec148b3a-c4f7-11ef-8ab8-84a938f0168c	88	461	Disney
17.	6642	20089	ef11ac44-c4f7-11ef-8ab8-84a938f0168c	6	589	Disney
18.	6643	20089	f20ebf20-c4f7-11ef-8ab8-84a938f0168c	23	631	Hulu
19.	6644	20089	f50bfef1-c4f7-11ef-8ab8-84a938f0168c	92	553	Netflix
20.	6645	20089	f809b092-c4f7-11ef-8ab8-84a938f0168c	64	172	Netflix
21.	6646	20089	fb06d526-c4f7-11ef-8ab8-84a938f0168c	4	827	Netflix
22.	6647	20089	fe04732c-c4f7-11ef-8ab8-84a938f0168c	79	476	Netflix

HDP.tlp - hdadmin@192.168.29.221:22 - Bitvise xterm - hdadmin@analytic: ~

```
1 row in set. Elapsed: 0.020 sec.  
analytic :) select count(1), CURRENT_TIMESTAMP() from movie_stream;  
SELECT  
count(1),  
CURRENT_TIMESTAMP()  
FROM movie_stream  
Query id: cfcf6572-8a2c-47a6-81fe-96f9d1802398
```

	count(1)	CURRENT_TIMESTAMP()
1.	14738	2024-12-29 11:33:47

1 row in set. Elapsed: 0.032 sec.
analytic :) select count(1), CURRENT_TIMESTAMP() from movie_stream;
SELECT
count(1),
CURRENT_TIMESTAMP()
FROM movie_stream
Query id: 6a2230f3-70d4-4358-8daf-3afe9ccb1e0a

	count(1)	CURRENT_TIMESTAMP()
1.	14742	2024-12-29 11:34:04

1 row in set. Elapsed: 0.021 sec.
analytic :)

Thank you!

For listening